

Dynamic Programming

1

Dynamic Programming

Strategy

- Dynamic Programming is a technique for solving optimization problems. Generally, such Problems have many feasible solutions with varying associated costs. The dynamic programming method determines the optimal solution, that is, the one which has the maximum or minimum cost.
- The algorithm works by splitting a problem into a sequence of optimization sub problems. The sub problems are solved and then combined to obtain solution to the main problem.
- The dynamic programming seems to have the approach of divide-and-conquer algorithm. There are, however, some subtle differences. Unlike the divide-and-conquer method, the dynamic programming splits the problem into overlapping sub problems. Further, the optimal solutions to sub problems are stored into a table, which is used to build the final or *global optimal* solution.

2

Dynamic Programming

Principle of Optimality

- Dynamic Programming is based on the principle of optimality. The principle states that optimal solution to a problem can be found if, there exists an optimal solution to a sub problem.
- For example, the principle of optimality holds in the case of problem of finding a shortest distance in a network, but it does not hold good for problem of finding the longest distance.
- Consider, for example, the network shown in the figure. The shortest distance A to D is $A \rightarrow B \rightarrow C \rightarrow D = 35$. Further, the shortest distance from A to C is $A \rightarrow B \rightarrow C = 25$, and shortest distance from A to B is $A \rightarrow B = 20$. Thus, principle of optimality holds true for each of the sub paths of the shortest distance between A to D.
- Now consider the longest distance from C to A. It is $C \rightarrow D \rightarrow A = 60$. However, the sub path $C \rightarrow D = 10$ is not the longest distance, because the longest distance from C to D is $C \rightarrow B \rightarrow A \rightarrow D = 75$. Hence, the principle of optimality does not hold true.

3

Dynamic Programming

Advantages and Disadvantage

The main advantages of dynamic programming are:

- Dynamic Programming can be used to solve optimization problems very efficiently, which cannot be solved efficiently by brute force and divide-and-conquer methods
- The implementation of dynamic programming solution is simple. The code consists of mainly nested loops which can be easily coded.
- The time and space complexity analysis of dynamic programming is easy and straightforward. Running time can be determined by counting loop cycles.

The disadvantages associated with dynamic programming are:

- There is an overhead of memory requirement to store tables for the solution of sub problems
- The formulation of sub problem structure is difficult
- The principle of optimality must hold.

4

Dynamic Programming

Applications

Some typical applications of dynamic programming are:

- ▪ *Computing Fibonacci Numbers*
- ▪ *Computing Binomial coefficients*
- ▪ *Matrix chain multiplication*
- ▪ *Finding shortest distance in a network*
- ▪ *Determining longest common subsequence*
- *Optimal triangulation of a polygon*
- *Optimized binary search trees*
- ▪ *Knapsack problem*
- *Optimal Assembly line scheduling*

5

Dynamic Programming

Steps

The dynamic programming solves a problem in four distinct steps::

- *A structure of the sub problem is identified*
- *A recursion is set up to obtain optimal solution to the sub problem*
- *The sub problem is solved in bottom-up manner, by using the previously computed values or boundary conditions. The results are saved in a table*
- *The solutions to sub problems are used to construct the global solution*

$$F(n) = F(n-1) + F(n-2)$$

$$F(0) = 1 \quad 2 \\ F(1) = 1 \quad -$$

6

Binomial Coefficients

Definition

- The **binomial coefficients** are a sequence of integers that arise in the expansion of the expression, $(x+y)^n$, often termed as **binomial theorem**. The expansion is written as

$$(x+y)^n = \sum_{k=0}^n c(n, k) x^k y^{n-k}, \quad n \geq 0$$

where $c(n, k)$ are the binomial coefficients.

- The coefficients are given, **in terms of factorials**, by the formula:

$$c(n, k) = \frac{n!}{k! (n-k)!}$$

- Since **factorial grows very rapidly**, exact computation of binomial coefficient by using the formula is not feasible even for moderately large n . The dynamic programming method provides an efficient algorithm for the computation.

7

Binomial Coefficients

Recursive Property

- The **binomial coefficients** have the following **recursive property**:

$$c(n, k) = c(n-1, k) + c(n-1, k-1)$$

with the **initial conditions**,

$$c(n, 0) = 1, \quad \text{since } 0! = 1 \text{ (By definition)}$$

$$c(0, k) = 0,$$

$$\text{and } c(n, k) = 0, \quad k > n,$$

since the coefficients are not defined for **values of k larger than n**

- The property can be established by using the basic formula:

8

Binomial Coefficients

Proof of Recursive Property

Consider $c(n-1, k)$,

$$c(n-1, k) = \frac{(n-1)!}{k! (n-k-1)!} \quad \text{(Using formula)}$$

$$= \frac{n-k}{n} \frac{n}{(n-k)} \frac{(n-1)!}{k! (n-k-1)!} \quad \text{(Multiplying and dividing by same factors } n, n-k)$$

$$= \frac{n-k}{n} \frac{n!}{k! (n-k)!} = \frac{n-k}{n} c(n, k) \quad \text{.....(I)}$$

Now consider $c(n-1, k-1)$,

$$c(n-1, k-1) = \frac{(n-1)!}{(k-1)! (n-k)!} \quad \text{(Using formula)}$$

$$= \frac{k}{n} \frac{n}{k} \frac{(n-1)!}{(k-1)! (n-k)!} \quad \text{(Multiplying and dividing by same factors } n, k)$$

$$= \frac{k}{n} \frac{n!}{k! (n-k)!} = \frac{k}{n} c(n, k) \quad \text{.....(II)}$$

Adding (I) and (II),

$$c(n-1, k) + c(n-1, k-1) = \left[\frac{n-k}{n} + \frac{k}{n} \right] c(n, k) = c(n, k),$$

Therefore,

$$c(n, k) = c(n-1, k) + c(n-1, k-1)$$

9

DP Solution for Binomial Coefficients

Pseudo Code

The recursion can be solved by using **dynamic programming (DP)**. The coefficients are computed **row-by-row**, using the values stored in the preceding rows and columns. The entries in the first row are obtained from the initial conditions.

- The pseudo code for the algorithm is as follows.

```

BINOMIAL(n, k)
  for i ← 1 to k do
    c[0, i] ← 0
  for i ← 0 to n do
    for j ← 0 to k do
      if j = 0 or j = i
        then c[i, j] ← 1
      else c[i, j] ← c[i-1, j] + c[i-1, j-1]
  
```

- The code for the algorithm consists of two nested loops. The outer loop executes n times and the inner loop k times. Thus, running time for the computation of binomial coefficients is $\theta(kn)$.

$(n+y)^m$ $F_1, \sqrt{m}$

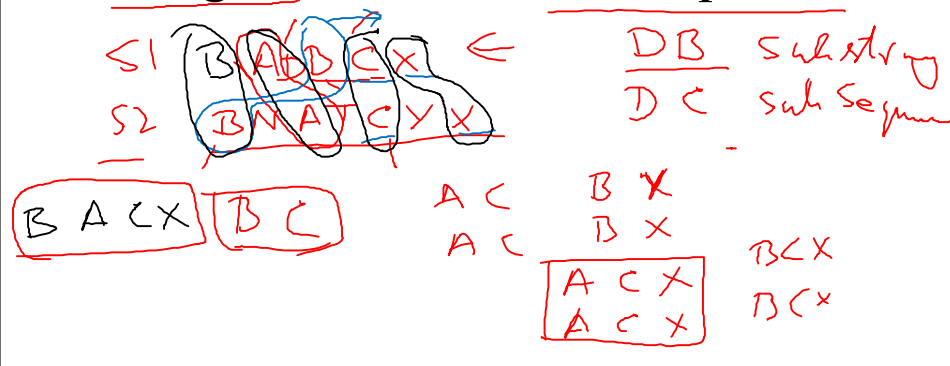
$\binom{n}{k}$ $c(n, k)$

	0	1	2	3	4
0	1	0	0	0	0
1	1	1	0	0	0
2	1	2	1	0	0
3	1	3	3	1	0
4	1	4	6	4	1
5	1	5	10	10	5

$c(5, 4)$

10

Longest Common Subsequence



11

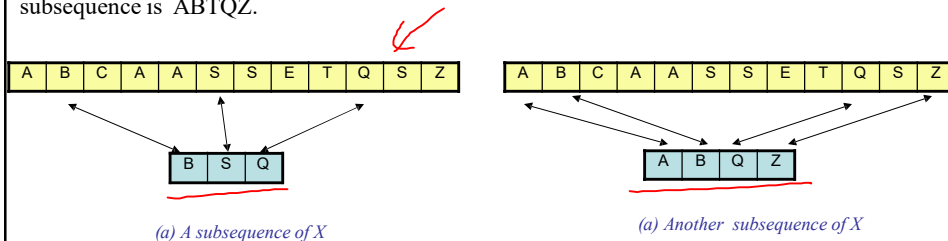
Longest Common Subsequence

subsequences

- Let X be a finite string of characters. A **subsequence** of X is enumeration of characters such that

- Characters in the sequence **appear in the same order** as in the string X
- The characters are not necessarily **consecutive**.

- Consider, for example, the string ABCAASSETQSZ. It has a subsequence BSQ. Another subsequence is ABTQZ.



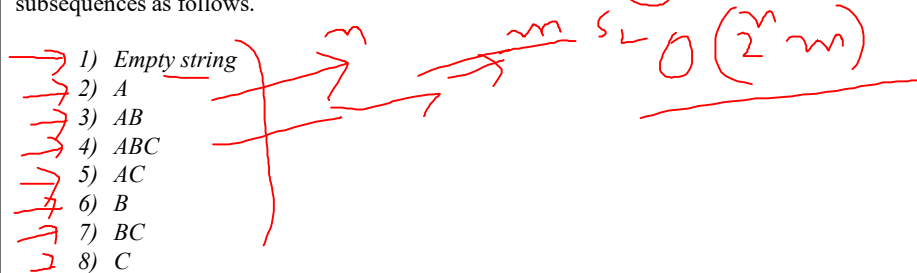
AXBYCN n 2^n
 TAGBNCN m $O(2^m)$

12

Longest Common Subsequence

Enumeration of subsequences

▪ A string of size n has 2^n subsequences. For example, the string ABC has eight subsequences as follows.



▪ The number subsequences can be evaluated by considering different way of filling a particular character position. In selecting a subsequence, a character from the source string is either included or dropped. Thus, position can be filled in two ways. Therefore, there are 2^n ways of filling n positions, which gives the total number of subsequences.

13

Longest Common Subsequence

Problem

Given two character strings $X = x_1x_2x_3 \dots x_n$, $Y = y_1y_2y_3 \dots y_m$, the problem is to find the largest common subsequence (LCS)

▪ The LCS problem arises in computational genetics to compare two strands of DNA. A strand consists of string of molecules called bases. There are four bases designated as A (adenine), G (guanine), C (cytosine), and T (thymine). For example the strand of one organism can, consist of string ACCGTCGGGGACTTAATTGGGGGCAATAA, and that of another strand can be AGGGAAATTGTGTAAACCACCATTA. Two organisms are stated to be genetically related if their strands have longest common subsequence

14

Longest Common Subsequence

Brute Force Algorithm

In **brute force** approach for finding longest common subsequence of two string X,Y, we can first find all the subsequences of the string X, and then compare each sequence with characters in string Y.

If there are n characters in string X and m characters in string Y, there would be 2^n subsequences of X. Thus, on the whole, $m \cdot 2^n$ operations would be needed to find the longest common sequence of X and Y. The brute force algorithm would have a running time of $\Theta(m2^n)$

15

Longest Common Subsequence

Dynamic Programming Formulation

In dynamic programming method, the character strings are compared together to count the number of common characters. The information about matches is stored in a table which gives the size of the longest subsequence, as well as the composition of the longest subsequence.

• Suppose X and Y are the given strings for which common subsequence is to be found. Let X_i and Y_j be the substrings of sizes i, j . Let $c(i, j)$ be the **length** of common of strings X_i, Y_j subsequence. Let x_i, y_j be the last characters in the strings. The following possibilities arise for computing $c(i, j)$ by using the preceding values :

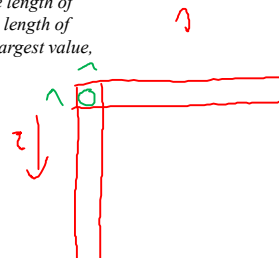
→ (i) If $x_i = y_j$, then the last common character in X_i and Y_j would be added to the count of longest common subsequence. This means that $c(i, j)$ would be obtained from $c(i-1, j-1)$ by adding one i.e. $c(i, j) = c(i-1, j-1) + 1$

→ (ii) If $x_i \neq y_j$ then last characters do not match. In this case $c(i-1, j)$ holds the length of longest common subsequence in strings X_{i-1} and Y_j , and $c(i, j-1)$ holds the length of longest common subsequence in strings X_i, Y_{j-1} . Since, $c(i, j)$ is to hold the largest value, $c(i, j)$ would be set equal to maximum of $c(i-1, j)$ and $c(i, j-1)$

(iii) Finally, if X_i is empty then $c(i, 0) = 0$, and if Y_j is empty then $c(0, j) = 0$

• It follows that $c(i, j)$ has the recurrence

$$c(i, j) = \begin{cases} 0, & i=0 \text{ or } j=0 \\ c(i-1, j-1) + 1, & \text{if } i, j > 0, \text{ and } x_i = y_j \\ \max(c(i, j-1), c(i-1, j)), & \text{if } x_i \neq y_j \end{cases}$$



16

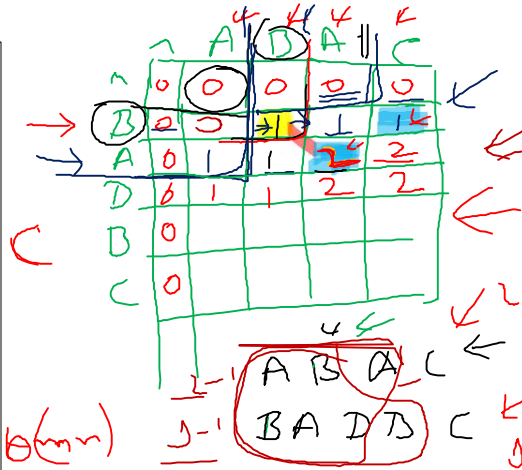
Longest Common Subsequence

Pseudo Code

The following code computes the length of longest common subsequence of two character strings X, Y . Array $b[1..n, 1..m]$ stores information about the optimal values of the subproblems. It is later used to construct the longest common sequence. An entry 'D' refers to diagonal cell, entry 'U' refers to cell above and entry 'L' refers to cell on the left.

```

LCS-LENGTH( $X, Y$ )
 $m \leftarrow \text{length}[X]$ 
 $n \leftarrow \text{length}[Y]$ 
for  $i \leftarrow 1$  to  $m$  do
     $c[i, 0] \leftarrow 0$ 
for  $j \leftarrow 1$  to  $n$  do
     $c[0, j] \leftarrow 0$ 
for  $i \leftarrow 1$  to  $m$  do
    for  $j \leftarrow 1$  to  $n$  do
        if  $x_i = y_j$ 
            then  $c[i, j] \leftarrow c[i-1, j-1] + 1$ 
             $b[i, j] \leftarrow \text{"D"}$ 
        else if  $c[i-1, j] \geq c[i, j-1]$ 
            then  $c[i, j] \leftarrow c[i-1, j]$ 
             $b[i, j] \leftarrow \text{"U"}$ 
        else
             $c[i, j] \leftarrow c[i, j-1]$ 
             $b[i, j] \leftarrow \text{"L"}$ 
    
```



17

Longest Common Subsequence

Pseudo Code

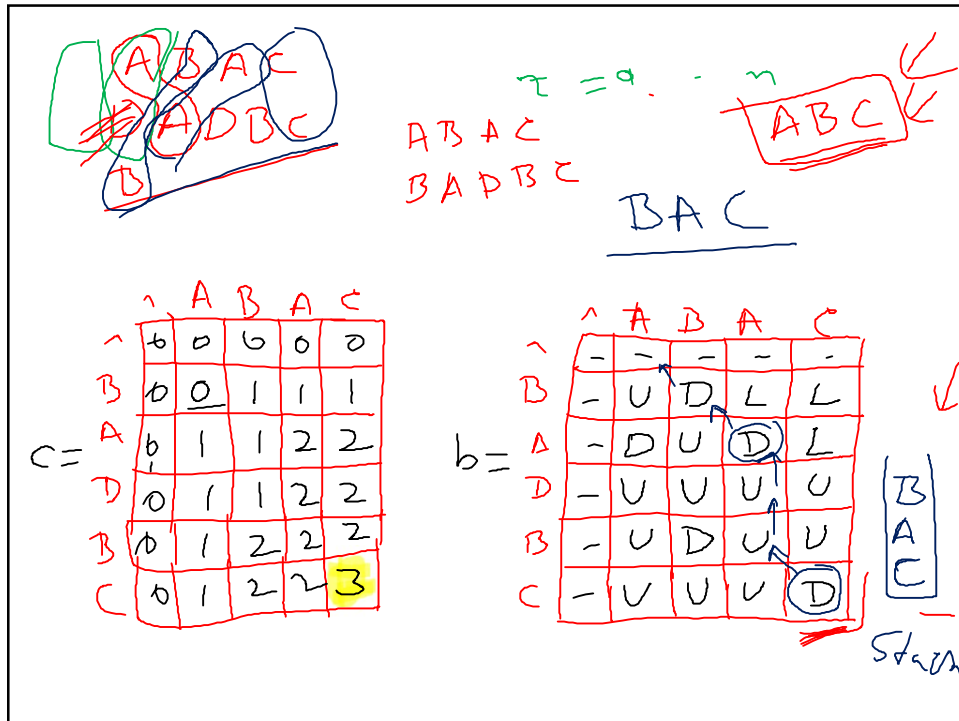
The following code prints the longest common subsequence. Using the indicators, D, U, and L, the order table is searched for the matching characters.

```

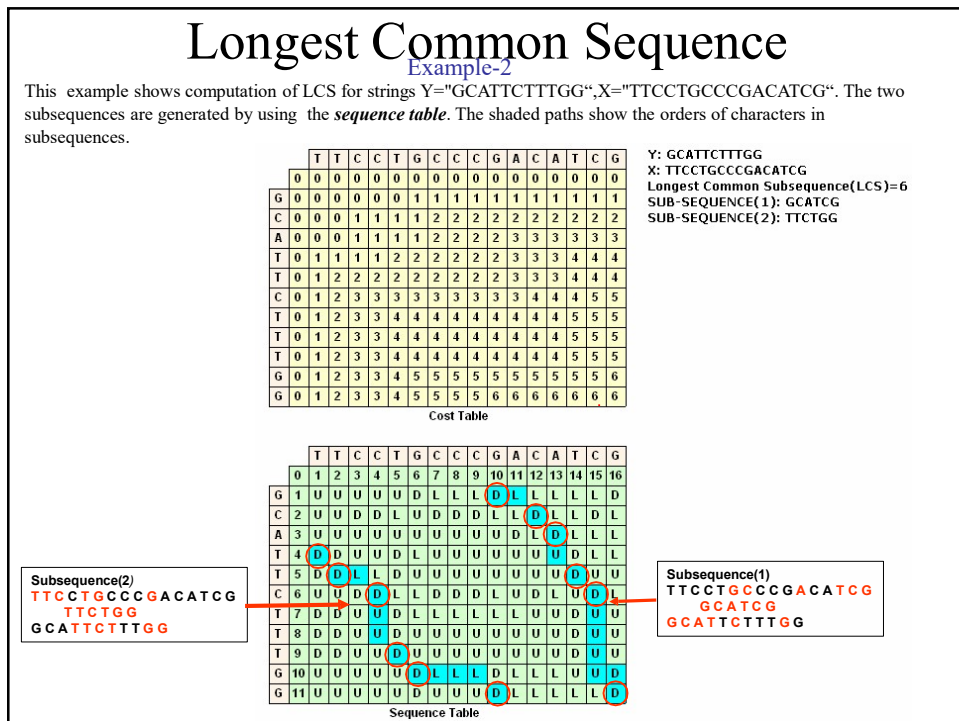
PRINT-LCS ( $b, X, i, j$ )
if  $i=0$  or  $j=0$ 
    then return
if  $b[i, j] = \text{"D"}$ 
    then PRINT-LCS( $b, X, i-1, j-1$ )
else if  $b[i, j] = \text{"U"}$ 
    then PRINT-LCS( $b, X, i-1, j$ )
else PRINT-LCS( $b, X, i, j-1$ )
    
```

~~Handwritten signature~~

18



19



20

Longest Common Subsequence

Analysis

Let $T(n, m)$ be the running time to compute the common subsequence of two strings of lengths n, m . From the pseudo code for computing the LCS, it follows that the running time is dominated by **two nested loops**. One loop iterates n times and the second loop iterates m times. The total time for execution of the loops is of the order of $m \times n$. Thus,

$$T(n, m) = \theta(m \cdot n)$$

21

Longest Common Subsequence

Comparison

• Let $T_{brute}(n, m)$ be the running time to compute the common subsequence of two strings of lengths n, m using brute force approach. Let $T_{dp}(n, m)$ be the corresponding running time for dynamic programming technique. Then,

$$\rightarrow T_{brute}(n, m) \approx m \cdot 2^n$$
$$T_{dp}(n, m) \approx m \cdot n$$

• The table summarizes the running times for the two algorithms, when one of the strings has length of 20. It follows that brute force method is **highly inefficient** compared to dynamic programming.

n	$T_{brute}(n, m)$	$T_{dp}(n, m)$
10	20480	200
20	20971520	400
30	21474836480	600
40	21990232555520	800
50	2.25×10^{16}	1000

22